

Package ‘fofrCVS’

August 14, 2026

Type Package

Title Control-Variates Transfer Learning for Function-on-Function Regression

Version 1.0.0

Description Implements target-only, feasible control-variates (CVS), and penalized control-variates (pCVS) estimators for function-on-function linear regression with multiple functional predictors and scalar covariates. Local models are represented in common predictor and response bases and fitted by penalized least squares. Study-level coefficient, plug-in mean, and covariance summaries can be combined without transferring subject-level observations, while a group penalty in the integrated coefficient-function geometry protects against negative transfer. The package provides centralized and distributed fitting interfaces, coefficient extraction and reconstruction, prediction, numerical diagnostics, and simulation-based data generation.

Depends R (>= 3.5.0)

Imports splines,
stats

License GPL-3

Encoding UTF-8

ByteCompile true

Contents

coef.cvs_fofr_fit	2
coefficient_surface	4
combine_cvs_fofr	6
fit_cvs_fofr	10
fit_local_fofr	14
fofr_basis	18
fofr_site_summary	20
predict.cvs_fofr_fit	22
print.cvs_fofr_fit	25
scalar_effect	26
simulate_fofr_data	29

Index	33
--------------	-----------

coef.cvs_fofr_fit *Extract Basis Coefficients from a CVS-FoFR Fit*

Description

Extracts the tensor-product basis coefficient matrix associated with the Target-only, control-variates (CVS), or penalized control-variates (pCVS) estimator from a fitted function-on-function transfer model.

Usage

```
## S3 method for class cvs_fofr_fit
coef(
  object,
  method = c("local", "CVS", "pCVS"),
  ...
)
```

Arguments

object	An object of class "cvs_fofr_fit", as returned by fit_cvs_fofr or combine_cvs_fofr .
method	A character string specifying the estimator to extract. "local" returns the Target-only estimator, "CVS" returns the feasible control-variates estimator, and "pCVS" returns the selected penalized control-variates estimator. Partial matching is supported.
...	Additional arguments for the coef generic. They are currently unused.

Details

Let P denote the number of functional predictors, Q the number of scalar covariates, and M_x and M_y the predictor- and response-basis dimensions. The fitted coefficient object is represented by a matrix $\hat{\mathbf{\Gamma}}$ of dimension $L \times M_y$, where $L = PM_x + Q$. Its first PM_x rows form the functional blocks $\hat{\mathbf{\Gamma}}_p \in \mathbb{R}^{M_x \times M_y}$, and its final Q rows form the scalar block $\hat{\mathbf{\Gamma}}_w \in \mathbb{R}^{Q \times M_y}$.

For common predictor and response bases $\phi(s)$ and $\vartheta(t)$, respectively, the fitted coefficient functions are reconstructed as

$$\hat{\beta}_p(s, t) = \phi(s)^\top \hat{\mathbf{\Gamma}}_p \vartheta(t),$$

and

$$\hat{\alpha}(t) = \hat{\mathbf{\Gamma}}_w \vartheta(t).$$

Thus, the matrix returned by this method contains coefficients in the common finite-dimensional sieve; it does not contain evaluations of the coefficient functions on a grid.

For method = "local", the matrix is the penalized Target-only estimator. For method = "CVS", it is obtained by reshaping the feasible control-variates coefficient vector

$$\hat{\mathbf{g}}_C = \hat{\mathbf{g}}_0 - \hat{\mathbf{U}}_* \left(\hat{\mathbf{\Delta}} - \hat{\mathbf{d}} \right),$$

where the adjustment combines centered target–source coefficient differences using the estimated study-level covariance matrices. For method = "pCVS", the returned matrix corresponds to the selected point on the pCVS path. The latter estimates the source-specific discrepancies by a group

lasso after transformation by the square root of the coefficient Gram matrix. Consequently, the group penalty measures integrated coefficient distance and is invariant to rescaling of the basis.

The stored estimator is vectorized and reshaped columnwise, consistently with the convention $\mathbf{g} = \text{vec}(\mathbf{\Gamma})$ used in the transfer construction. Extraction with method = "pCVS" requires that a pCVS solution has been selected, either at a user-supplied penalty or by a target validation sample; otherwise the method returns an error.

The numerical entries of $\hat{\mathbf{\Gamma}}$ depend on the chosen bases. Interpretation and comparison of fitted effects should therefore usually be based on [coefficient_surface](#) or [scalar_effect](#), which reconstruct the corresponding functions. The row names retain the fitted predictor and covariate names so that the block structure can be identified without consulting the original data.

Value

A numeric matrix with $PM_x + Q$ rows and M_y columns. For each functional predictor, M_x consecutive rows contain its predictor-basis coefficients; the final Q rows correspond to the scalar covariates. The columns index the response-basis functions. The matrix has an attribute named "method" recording the extracted estimator.

Author(s)

Ufuk Beyaztas, Muge Mutis, Sude Gurer and Soutir Bandyopadhyay.

Examples

```
s_grid <- seq(0, 1, length.out = 101)
t_grid <- seq(0, 1, length.out = 101)

target <- simulate_fofr_data(40, s_grid = s_grid, t_grid = t_grid)
source <- simulate_fofr_data(80, s_grid = s_grid, t_grid = t_grid)
validation <- simulate_fofr_data(40, s_grid = s_grid, t_grid = t_grid)

fit <- fit_cvs_fofr(
  datasets = list(target, source),
  x_basis_type = "bspline",
  y_basis_type = "bspline",
  Mx = 5,
  My = 5,
  variance_method = "model",
  target_validation = validation
)

gamma_target <- coef(fit, method = "local")
gamma_cvs <- coef(fit, method = "CVS")
gamma_pcv <- coef(fit, method = "pCVS")

dim(gamma_pcv)
head(gamma_pcv)
attr(gamma_pcv, "method")
```

coefficient_surface *Evaluate a Fitted Function-on-Function Coefficient Surface*

Description

Reconstructs a bivariate coefficient surface for one functional predictor from the Target-only, control-variates (CVS), or penalized control-variates (pCVS) estimator in a fitted function-on-function transfer model.

Usage

```
coefficient_surface(
  object,
  predictor = 1,
  method = c("local", "CVS", "pCVS"),
  s_grid = NULL,
  t_grid = NULL
)
```

Arguments

object	An object of class "cvs_fofr_fit", as returned by fit_cvs_fofr or combine_cvs_fofr .
predictor	The functional predictor whose coefficient surface is to be evaluated. Supply either a single integer between 1 and the number of functional predictors or the exact predictor name recorded in object.
method	A character string specifying the estimator. "local" uses the Target-only estimator, "CVS" uses the feasible control-variates estimator, and "pCVS" uses the selected penalized control-variates estimator. Partial matching is supported.
s_grid	A numeric vector of predictor-domain locations at which the surface is to be evaluated. If NULL, 101 equally spaced points over the boundary of the fitted predictor basis are used.
t_grid	A numeric vector of response-domain locations at which the surface is to be evaluated. If NULL, 101 equally spaced points over the boundary of the fitted response basis are used.

Details

For the p th functional predictor, the function-on-function linear model contains the term

$$\int_S X_{ip}(s) \beta_p(s, t) ds,$$

so $\beta_p(s, t)$ describes how the predictor value at location s contributes to the response at location t . The fitted model represents this surface in a tensor-product sieve. If $\phi(s)$ is the predictor basis, $\vartheta(t)$ is the response basis, and $\hat{\mathbf{\Gamma}}_p$ is the coefficient block associated with predictor p , the reconstructed surface is

$$\hat{\beta}_p(s, t) = \phi(s)^\top \hat{\mathbf{\Gamma}}_p \vartheta(t).$$

The function first selects the requested Target-only, CVS, or pCVS coefficient vector, reshapes it into the fitted coefficient matrix, and extracts the $M_x \times M_y$ block belonging to predictor. It

then evaluates the two bases on `s_grid` and `t_grid` and performs the tensor-product reconstruction above.

The supplied grids are evaluation grids only. They may differ from the grids on which the functional observations were recorded, and changing them does not refit the model. For the usual interpretation, their values should lie within the boundaries of the fitted predictor and response bases. The returned surface has one row for each value of `s_grid` and one column for each value of `t_grid`; explicitly, `beta[i, j]` is the estimate at `(s_grid[i], t_grid[j])`.

Although the numerical basis coefficients depend on the chosen bases, the reconstructed coefficient surface is the functional object represented by those coefficients. In particular, the Gram-metric formulation used by pCVS ensures that its discrepancy penalty measures integrated surface distance rather than an arbitrary Euclidean distance between basis coefficients. Extraction with `method = "pCVS"` requires a selected pCVS solution; otherwise an error is returned.

Value

A list with the following components:

<code>s_grid</code>	The predictor-domain evaluation grid.
<code>t_grid</code>	The response-domain evaluation grid.
<code>beta</code>	A numeric matrix of dimension <code>length(s_grid)</code> by <code>length(t_grid)</code> . Entry <code>beta[i, j]</code> equals the reconstructed coefficient surface at <code>(s_grid[i], t_grid[j])</code> .
<code>method</code>	The estimator used for reconstruction.
<code>predictor</code>	The numerical index of the selected functional predictor.

Author(s)

Ufuk Beyaztas, Muge Mutis, Sude Gurer and Soutir Bandyopadhyay.

Examples

```
s_grid <- seq(0, 1, length.out = 101)
t_grid <- seq(0, 1, length.out = 101)

target <- simulate_fofr_data(40, s_grid = s_grid, t_grid = t_grid)
source <- simulate_fofr_data(80, s_grid = s_grid, t_grid = t_grid)
validation <- simulate_fofr_data(40, s_grid = s_grid, t_grid = t_grid)

fit <- fit_cvs_fofr(
  datasets = list(target, source),
  x_basis_type = "bspline",
  y_basis_type = "bspline",
  Mx = 5,
  My = 5,
  variance_method = "model",
  target_validation = validation
)

surface <- coefficient_surface(
  fit,
  predictor = "activity",
  method = "pCVS"
```

```

)

dim(surface$beta)
surface$beta[1:3, 1:3]

persp(
  surface$s_grid,
  surface$t_grid,
  surface$beta,
  theta = 35,
  phi = 25,
  expand = 0.6,
  shade = 0.4,
  col = "lightblue",
  ticktype = "detailed",
  xlab = "s",
  ylab = "t",
  zlab = expression(hat(beta)(s, t))
)

```

combine_cvs_fofr

Combine Local Fits by CVS and pCVS Transfer

Description

Combines a target-site function-on-function fit with one or more source-site summaries. The function computes the Target-only and feasible control-variates (CVS) estimators, constructs the penalized control-variates (pCVS) solution path, and optionally selects a pCVS estimator using an independent target validation sample or a user-specified penalty.

Usage

```

combine_cvs_fofr(
  target_fit,
  source_summaries,
  zeta = NULL,
  zeta_grid = NULL,
  target_validation = NULL,
  tolerance = 1e-7,
  max_iterations = 20000
)

```

Arguments

target_fit	An object of class "local_fofr_fit" obtained by applying fit_local_fofr to the target study.
source_summaries	A nonempty list of source-site summaries produced by fofr_site_summary . Objects of class "local_fofr_fit" are also accepted and are converted internally. All studies must use identical predictor and response bases and the same ordered functional predictors and scalar covariates.

zeta	An optional nonnegative finite number specifying a pCVS penalty. The value is added to the candidate path. If <code>target_validation = NULL</code> , the pCVS estimate at this value is selected. If validation data are supplied, zeta is treated as an additional candidate and validation selection takes precedence.
zeta_grid	An optional numeric vector of nonnegative finite pCVS penalties. If <code>NULL</code> , a decreasing data-adaptive path is constructed from the source-discrepancy gradients. The exact Target-only endpoint <code>zeta = 0</code> is always included, duplicate values are removed, and the resulting path is sorted in decreasing order.
target_validation	An optional independent target validation data set. It must be a list with components <code>Y</code> , <code>y_grid</code> , <code>X_list</code> , and <code>x_grids</code> , and with <code>w</code> when scalar covariates are present. The pCVS penalty is selected by integrated squared prediction error on these response curves.
tolerance	A positive finite convergence tolerance for the accelerated proximal-gradient algorithm used along the pCVS path.
max_iterations	A positive number giving the maximum number of proximal-gradient iterations at each value of zeta.

Details

Each study is represented by a coefficient vector $\hat{g}_k$, a plug-in conditional mean $\hat{\mu}_k$, an estimated covariance matrix $\hat{\Sigma}_k$, and a basis signature. Let study 0 denote the target and let studies $1, \dots, K$ denote the sources. The stacked target–source differences are

$$\hat{\Delta} = \{(\hat{g}_0 - \hat{g}_1)^\top, \dots, (\hat{g}_0 - \hat{g}_K)^\top\}^\top,$$

with estimated mean $\hat{d}$ formed analogously from the exported plug-in means. Writing

$$\hat{\Sigma}_* = \left(\sum_{k=0}^K \hat{\Sigma}_k^{-1} \right)^{-1}$$

and

$$\hat{U}_* = \hat{\Sigma}_* \left[\hat{\Sigma}_1^{-1}, \dots, \hat{\Sigma}_K^{-1} \right],$$

the feasible control-variates estimator is

$$\hat{g}_C = \hat{g}_0 - \hat{U}_*(\hat{\Delta} - \hat{d}).$$

This construction combines precision-weighted, centered local fluctuations; it does not pool subject-level observations or require the source regression operators to equal the target operator.

The pCVS estimator regularizes the source discrepancies in the functional geometry of the coefficient object. Let $G = \Psi_y \otimes \text{diag}(\Psi_x, \dots, \Psi_x, I_Q)$ be the coefficient Gram matrix and define metric coordinates $\hat{\theta}_k = G^{1/2} \hat{g}_k$. For a stacked candidate discrepancy $u = (u_1^\top, \dots, u_K^\top)^\top$, the path solves

$$\hat{d}_\zeta = \arg \min_u \left\{ (\hat{\Delta}_\theta - u)^\top Q_{\text{alg}} (\hat{\Delta}_\theta - u) + \zeta \sum_{k=1}^K \|u_k\|_2 \right\}.$$

The corresponding coefficient estimator is

$$\hat{\theta}_{PC}(\zeta) = \hat{\theta}_0 - \hat{U}_{\theta*}(\hat{\Delta}_\theta - \hat{d}_\zeta).$$

Because multiplication by $G^{1/2}$ is an isometry, each group norm is the integrated Hilbert–Schmidt distance between the target and one source coefficient object, rather than a basis-dependent Euclidean norm.

At $\zeta = 0$, the discrepancy estimate equals the observed discrepancy and pCVS reduces exactly to the Target-only estimator. At sufficiently large penalties, the discrepancy blocks are zero and the sources enter through the zero-discrepancy control-variate combination. Between these endpoints, a source with a discrepancy block near zero transfers strongly, whereas a block retained near its observed target–source difference largely withdraws that source. The entire path is computed with warm starts. A solution is eligible for validation selection only if both the convergence and Karush–Kuhn–Tucker criteria are satisfied.

When `target_validation` is supplied, the selected penalty minimizes

$$\frac{1}{n_v} \sum_{i=1}^{n_v} \int_{\mathcal{T}} \{Y_i(t) - \hat{Y}_{i,\zeta}(t)\}^2 dt$$

over the converged path solutions, using trapezoidal integration on the validation response grid. If neither `target_validation` nor ζ is supplied, the CVS estimator and the full pCVS path are returned but no pCVS coefficient vector is selected; consequently, `gamma_pcv`s and `selected_zeta` are NULL.

Before combination, the function verifies dimensions, basis definitions, Gram and penalty matrices, and the ordered predictor names through the basis signatures. Covariance inverses and discrepancy precisions are spectrally stabilized when required. Nonconvergent path points are retained for diagnostic purposes but are assigned infinite validation loss and cannot be selected.

Value

An object of class `"cvs_fofr_fit"`. It is a list containing:

<code>target_fit</code>	The target-site object supplied in <code>target_fit</code> .
<code>source_summaries</code>	The validated source summaries used in transfer.
<code>gamma_local</code>	The Target-only coefficient vector of length d .
<code>gamma_cvs</code>	The feasible CVS coefficient vector of length d .
<code>gamma_pcv</code>	The selected pCVS coefficient vector of length d , or NULL when no pCVS penalty has been selected.
<code>U_star</code>	The $d \times Kd$ control-variate weight matrix in the original basis coordinates.
<code>U_star_metric</code>	The $d \times Kd$ control-variate weight matrix in Gram-metric coordinates.
<code>delta_hat</code>	The stacked observed target–source coefficient differences in the original basis coordinates.
<code>expected_delta</code>	The corresponding stacked differences between the exported plug-in means.
<code>precision_delta</code>	The stabilized precision matrix of the stacked differences in the original basis coordinates.
<code>pooled_variance</code>	The precision-weighted aggregate covariance matrix in the original basis coordinates.
<code>functional_metric</code>	The $d \times d$ Gram matrix defining integrated coefficient distance.
<code>metric_root</code>	A symmetric square root of <code>functional_metric</code> .

metric_inverse_root	The corresponding spectrally stabilized inverse square root.
delta_hat_metric	The stacked observed target–source differences in Gram-metric coordinates.
expected_delta_metric	The stacked plug-in mean differences in Gram-metric coordinates.
precision_delta_metric	The stabilized discrepancy precision used by the pCVS optimization.
pooled_variance_metric	The precision-weighted aggregate covariance matrix in Gram-metric coordinates.
zeta_grid	The decreasing sequence of pCVS penalties that was fitted.
delta_path	A matrix whose rows are the estimated discrepancy vectors along the pCVS path, expressed in the original basis coordinates.
delta_path_metric	The corresponding discrepancy path in Gram-metric coordinates.
pcvs_coefficient_path	A matrix whose rows contain the pCVS coefficient vectors in original basis coordinates, in the order of zeta_grid.
validation_mse	Integrated validation curve MSE at every path point; a vector of NA_real_ values when validation data were not supplied.
selected_zeta	The selected pCVS penalty, or NULL.
selected_index	The index of selected_zeta in zeta_grid, or NULL.
selection	A character string describing the selection rule.
convergence	A logical vector recording convergence of every pCVS path solution.
iterations	The number of iterations used at every path point.
kkt_residual	The absolute KKT residual at every path point.
relative_kkt_residual	The scale-adjusted KKT residual at every path point.
K	The number of source studies.
group_size	The coefficient dimension $d = (PM_x + Q)M_y$ associated with each source group.

Author(s)

Ufuk Beyaztas, Muge Mutis, Sude Gurer and Soutir Bandyopadhyay.

Examples

```
s_grid <- seq(0, 1, length.out = 101)
t_grid <- seq(0, 1, length.out = 101)

target <- simulate_fofr_data(40, s_grid = s_grid, t_grid = t_grid)
source <- simulate_fofr_data(80, s_grid = s_grid, t_grid = t_grid)
validation <- simulate_fofr_data(40, s_grid = s_grid, t_grid = t_grid)

x_basis <- fofr_basis("bspline", M = 5)
```

```

y_basis <- fofr_basis("bspline", M = 5)

target_fit <- fit_local_fofr(
  target,
  x_basis = x_basis,
  y_basis = y_basis,
  variance_method = "model"
)
source_fit <- fit_local_fofr(
  source,
  x_basis = x_basis,
  y_basis = y_basis,
  variance_method = "model"
)

# Only the aggregate source summary is needed by the target site.
source_summary <- fofr_site_summary(source_fit)

fit <- combine_cvs_fofr(
  target_fit = target_fit,
  source_summaries = list(source_summary),
  target_validation = validation
)

fit$selected_zeta
fit$convergence[fit$selected_index]
coef(fit, method = "CVS")[1:4, 1:3]
coef(fit, method = "pCVS")[1:4, 1:3]

```

fit_cvs_fofr

Fit CVS and pCVS Function-on-Function Transfer Models

Description

Fits the complete control-variates transfer procedure for function-on-function linear regression with multiple functional predictors and optional scalar covariates. The first data set is treated as the target study and all remaining data sets as sources. The returned object contains the Target-only, feasible control-variates (CVS), and penalized control-variates (pCVS) estimators.

Usage

```

fit_cvs_fofr(
  datasets,
  x_basis_type = c("fourier", "bspline"),
  y_basis_type = c("fourier", "bspline"),
  Mx = 7,
  My = 5,
  degree = 3,
  x_boundary = c(0, 1),
  y_boundary = c(0, 1),
  rho_x = NULL,
  rho_y = NULL,

```

```

lambda = NULL,
lambda_grid = NULL,
variance_method = c("HC", "model"),
zeta = NULL,
zeta_grid = NULL,
target_validation = NULL,
tolerance = 1e-7,
max_iterations = 20000
)

```

Arguments

datasets	A list containing the target data set first and at least one source data set thereafter. Each data set must contain <code>Y</code> , <code>y_grid</code> , <code>X_list</code> , and <code>x_grids</code> , and may contain a scalar-covariate matrix <code>w</code> ; see Details.
x_basis_type	The basis used over the functional-predictor domain: "fourier" or "bspline". Partial matching is supported.
y_basis_type	The basis used over the response domain: "fourier" or "bspline". Partial matching is supported.
Mx	The number of predictor-basis functions. It must be at least 3 and, for a B-spline basis, at least degree + 1.
My	The number of response-basis functions. It must be at least 3 and, for a B-spline basis, at least degree + 1.
degree	The polynomial degree used for both B-spline bases. It must be an integer of at least 2. This argument does not affect a Fourier basis.
x_boundary	A length-two increasing numeric vector defining the common functional-predictor domain. Every predictor grid in every study must lie within this interval.
y_boundary	A length-two increasing numeric vector defining the common response domain. Every response grid in every study must lie within this interval.
rho_x	Predictor-curve smoothing penalty. Supply NULL, one nonnegative number used for all functional predictors, or a nonnegative vector with one value per functional predictor. If NULL, a penalty is selected separately for each predictor in each study by generalized cross-validation.
rho_y	Response-curve smoothing penalty. Supply a single nonnegative number or NULL. If NULL, it is selected separately in each study by generalized cross-validation.
lambda	The nonnegative roughness penalty for the function-on-function coefficient surfaces. If supplied, the same value is used in every local study. If NULL, each study selects its own value by generalized cross-validation over <code>lambda_grid</code> .
lambda_grid	An optional nonempty numeric vector of nonnegative candidate values for <code>lambda</code> . It is used only when <code>lambda = NULL</code> . If both arguments are NULL, a data-scaled logarithmic grid is constructed separately for each study.
variance_method	Method used to estimate the conditional covariance of each local coefficient vector. "HC" gives a heteroskedasticity-consistent residual outer-product estimator with an effective-degrees-of-freedom correction. "model" uses a common residual covariance augmented by a correction for functional-predictor reconstruction noise. Partial matching is supported.

zeta	An optional nonnegative finite pCVS penalty. It is added to the candidate path and is selected when <code>target_validation = NULL</code> . If validation data are provided, it is treated as an additional candidate and validation selection takes precedence.
zeta_grid	An optional numeric vector of nonnegative finite pCVS penalties. If <code>NULL</code> , a decreasing data-adaptive path is constructed. The exact Target-only endpoint <code>zeta = 0</code> is always included.
target_validation	An optional independent target validation data set with the same component structure as an element of <code>datasets</code> . It is used only to select the pCVS penalty by integrated squared prediction error and is not included in any local fit.
tolerance	A positive finite convergence tolerance for the accelerated proximal-gradient algorithm used to compute the pCVS path.
max_iterations	A positive number giving the maximum number of proximal-gradient iterations at each pCVS penalty.

Details

The function accommodates the centered mixed-predictor model

$$Y_i^{(k)}(t) - \mu_Y^{(k)}(t) = \sum_{p=1}^P \int_S \{X_{ip}^{(k)}(s) - \mu_{X_p}^{(k)}(s)\} \beta_p^{(k)}(s, t) ds + \{W_i^{(k)} - \mu_W^{(k)}\}^\top \alpha^{(k)}(t) + \varepsilon_i^{(k)}(t),$$

where study 0 is the target and studies $1, \dots, K$ are sources. The coefficient surfaces and scalar-effect functions may differ between the target and the sources; equality of the regression operators is not imposed.

Each element of `datasets` is a study-level list. The response `Y` is an $n_k \times J_y^{(k)}$ matrix whose rows are subjects and whose columns correspond to the strictly increasing vector `y_grid`. The component `x_list` is a list of P matrices; its p th matrix has n_k rows and one column for each location in `x_grids[[p]]`. The optional `w` is an $n_k \times Q$ matrix. All entries must be finite and each study must contain at least five subjects. The numbers and ordered names of the functional predictors and scalar covariates must agree across studies. Observation grids may differ by study and predictor, provided that they remain inside the common basis boundaries.

The same predictor and response bases are constructed once and used in every study. Within each study, the recorded predictor and response curves are first smoothed onto these bases. If Φ_p and Θ are the corresponding evaluation matrices, the coefficient smoothers have the form

$$P_{xp} = (\Phi_p^\top \Phi_p + \rho_{xp} W_x)^{-1} \Phi_p^\top, \quad P_y = (\Theta^\top \Theta + \rho_y W_y)^{-1} \Theta^\top.$$

The resulting local coefficient matrix minimizes

$$\text{tr}\{(D_k - H_k \Gamma) \Psi_y (D_k - H_k \Gamma)^\top\} + \lambda_k \text{tr}(\Gamma^\top B \Gamma \Psi_y),$$

where B applies the second-derivative penalty to the functional blocks and leaves the scalar-effect block unpenalized. Writing $A_k = H_k^\top H_k + \lambda_k B$, the fitted matrix is

$$\hat{\Gamma}_k = A_k^{-1} H_k^\top D_k.$$

The implementation uses spectrally stabilized inverses when a fitted matrix is numerically singular or nearly singular and records local rank and condition diagnostics in `local_fits`.

After fitting all studies, the function extracts each local coefficient estimate, plug-in conditional mean, covariance estimate, and basis signature, then calls `combine_cvs_fofr`. The Target-only

estimator is the coefficient estimate from `datasets[[1]]`. CVS subtracts an optimally weighted centered target–source difference. pCVS estimates source-specific mean discrepancies by a group lasso after transformation by the coefficient Gram matrix. In that metric, the penalty is the integrated Hilbert–Schmidt distance between coefficient objects and does not depend on arbitrary scaling of the bases.

The pCVS path always contains $\text{zeta} = 0$, at which the estimator is exactly Target-only. When `target_validation` is supplied, only converged path solutions satisfying the Karush–Kuhn–Tucker tolerance are eligible, and the value minimizing integrated validation curve MSE is selected. When no validation sample is supplied, zeta must be specified to select a pCVS estimator. If both `target_validation` and zeta are NULL, the path is returned but `gamma_pcv` is NULL.

Value

An object of class `"cvs_fofr_fit"`. It contains all components documented for `combine_cvs_fofr`, including the Target-only, CVS, and selected pCVS coefficient vectors; the pCVS coefficient and discrepancy paths; the coefficient Gram metric and control-variate weights; validation losses; and convergence, iteration, and KKT diagnostics. In addition, this interface returns:

<code>local_fits</code>	A list of <code>"local_fofr_fit"</code> objects in the same order as <code>datasets</code> . The first element is the target fit and the remaining elements are the source fits.
<code>call</code>	The matched function call.

The principal coefficient components are `gamma_local`, `gamma_cvs`, and `gamma_pcv`. The last is NULL when no pCVS penalty was selected.

Author(s)

Ufuk Beyaztas, Muge Mutis, Sude Gurer and Soutir Bandyopadhyay.

Examples

```
s_grid <- seq(0, 1, length.out = 101)
t_grid <- seq(0, 1, length.out = 101)

target <- simulate_fofr_data(35, s_grid = s_grid, t_grid = t_grid)
source_1 <- simulate_fofr_data(60, s_grid = s_grid, t_grid = t_grid)
source_2 <- simulate_fofr_data(
  60,
  s_grid = s_grid,
  t_grid = t_grid,
  effect = "shifted"
)
validation <- simulate_fofr_data(35, s_grid = s_grid, t_grid = t_grid)
new_target <- simulate_fofr_data(10, s_grid = s_grid, t_grid = t_grid)

fit <- fit_cvs_fofr(
  datasets = list(target, source_1, source_2),
  x_basis_type = "bspline",
  y_basis_type = "bspline",
  Mx = 5,
  My = 5,
  variance_method = "model",
  target_validation = validation
)
```

```

)

fit
fit$selected_zeta
coef(fit, method = "pCVS")[1:4, 1:3]

predicted_curves <- predict(fit, new_target, method = "pCVS")
dim(predicted_curves)

activity_surface <- coefficient_surface(
  fit,
  predictor = "activity",
  method = "pCVS"
)
dim(activity_surface$beta)

```

fit_local_fofr

Fit a Local Function-on-Function Regression Model

Description

Fits the study-specific component of the control-variates transfer procedure. The function reconstructs the observed curves in common predictor and response bases, estimates the mixed-predictor function-on-function regression model by penalized least squares, and constructs the coefficient mean and covariance summaries required for CVS and pCVS transfer.

Usage

```

fit_local_fofr(
  dataset,
  x_basis,
  y_basis,
  rho_x = NULL,
  rho_y = NULL,
  lambda = NULL,
  lambda_grid = NULL,
  variance_method = c("HC", "model")
)

```

Arguments

dataset	A study-level list containing Y , y_grid , X_list , and x_grids , and optionally a scalar-covariate matrix w ; see Details.
x_basis	A predictor-domain basis object created by fofr_basis .
y_basis	A response-domain basis object created by fofr_basis .
rho_x	Predictor-curve smoothing penalty. Supply NULL, one nonnegative number used for every functional predictor, or a nonnegative vector with one value per functional predictor. If NULL, a penalty is selected separately for each predictor by generalized cross-validation.
rho_y	Response-curve smoothing penalty. Supply a single nonnegative number or NULL. If NULL, it is selected by generalized cross-validation.

lambda	A single nonnegative roughness penalty for the function-on-function coefficient surfaces. If NULL, the value is selected by generalized cross-validation over lambda_grid.
lambda_grid	An optional nonempty numeric vector of nonnegative candidates for lambda. It is used only when lambda = NULL. If both are NULL, a data-scaled logarithmic grid is constructed internally.
variance_method	Method used to estimate the conditional covariance of the local coefficient vector. "HC" gives a heteroskedasticity-consistent estimator based on subject-specific residual outer products. "model" uses a common residual covariance augmented by a correction for predictor-reconstruction noise. Partial matching is supported.

Details

The response Y must be an $n \times J_y$ matrix whose rows are subjects and whose columns correspond to the strictly increasing vector y_grid . The component X_list must contain at least one functional-predictor matrix. Its p th element has n rows and one column for every location in the strictly increasing vector $x_grids[[p]]$. The optional component w is an $n \times Q$ matrix of scalar covariates. All observations and grid values must be finite, and at least five subjects are required. Names attached to X_list and the columns of w are retained in the fitted object; otherwise default names are assigned.

Let $\phi(s)$ and $\vartheta(t)$ denote the predictor and response bases, with dimensions M_x and M_y . Their Gram matrices are Ψ_x and Ψ_y , and their integrated second-derivative penalty matrices are W_x and W_y . Every predictor grid must lie inside the boundary of x_basis , and the response grid must lie inside that of y_basis .

The response, functional predictors, and scalar covariates are centered within the study. For predictor p , let Φ_p be the evaluation matrix of ϕ on $x_grids[[p]]$. If x_{ip} is the centered observed trajectory, its smoothed basis coefficients are

$$\hat{c}_{ip} = P_{xp} x_{ip}, \quad P_{xp} = (\Phi_p^T \Phi_p + \rho_{xp} W_x)^{-1} \Phi_p^T.$$

Similarly, if Θ evaluates ϑ on y_grid , the smoothed response coefficients are

$$\hat{\xi}_i = P_y y_i, \quad P_y = (\Theta^T \Theta + \rho_y W_y)^{-1} \Theta^T.$$

When a smoothing penalty is not supplied, it minimizes the usual generalized-cross-validation score based on the corresponding linear smoother.

Put $L = PM_x + Q$. The integrated design vector for subject i is

$$h_i = (\hat{c}_{i1}^T \Psi_x, \dots, \hat{c}_{iP}^T \Psi_x, w_i^T)^T,$$

where w_i is centered. Let H have rows h_i^T and let D have rows $\hat{\xi}_i^T$. The coefficient matrix $\Gamma \in \mathbb{R}^{L \times M_y}$ is partitioned into P functional blocks followed by a $Q \times M_y$ scalar block. The local criterion is

$$\text{tr}\{(D - H\Gamma)\Psi_y(D - H\Gamma)^T\} + \lambda \text{tr}(\Gamma^T B \Gamma \Psi_y),$$

where $B = \text{diag}(W_x, \dots, W_x, 0_{Q \times Q})$. Thus, roughness in the predictor direction is penalized for every coefficient surface, while the scalar-effect functions are unpenalized in this step. With $\Omega = H^T H$ and $A = \Omega + \lambda B$, the fitted matrix is

$$\hat{\Gamma} = A^{-1} H^T D.$$

If λ is not supplied, generalized cross-validation uses the integrated residual coefficient loss and effective degrees of freedom $\nu = \text{tr}(\mathbf{A}^{-1}\mathbf{\Omega})$.

In exact arithmetic, uniqueness of the penalized solution requires

$$\text{Null}(\mathbf{H}) \cap \text{Null}(\lambda\mathbf{B}) = \{\mathbf{0}\}.$$

The implementation reports the numerical rank and condition number of $\mathbf{H}$ and warns about deficient or severely ill-conditioned designs. Spectrally floored inverses keep such fits computable, but individual coefficient effects may remain weakly identified.

The transfer summary requires an estimate of the conditional mean and covariance of $\hat{\mathbf{g}} = \text{vec}(\hat{\mathbf{\Gamma}})$. The component `expected_gamma` is a study-computable plug-in mean: the fitted conditional response curves are reconstructed on the observation grid, smoothed through $\mathbf{P}_y$, and mapped back through the local normal equations. It should not in general be interpreted as the exact conditional expectation of `gamma_hat`; exactness requires additional working-model and tuning conditions.

For `variance_method = "HC"`, the covariance estimate has the form

$$\hat{\Sigma}_{HC} = \frac{n}{n - \nu} \sum_{i=1}^n (\hat{\mathbf{r}}_i \hat{\mathbf{r}}_i^{\top}) \otimes (\mathbf{a}_i \mathbf{a}_i^{\top}), \quad \mathbf{a}_i = \mathbf{A}^{-1} \mathbf{h}_i,$$

where $\hat{\mathbf{r}}_i$ is the smoothed residual coefficient vector. This option requires $n > \nu$; before spectral regularization it is rank deficient when $n < LM_y$. For `variance_method = "model"`, a common response covariance, augmented by the estimated predictor-reconstruction contribution, is combined with $\mathbf{A}^{-1} \mathbf{H}^{\top} \mathbf{H} \mathbf{A}^{-1}$ through a Kronecker product. When tuning parameters are selected using the same responses employed in fitting, these covariance formulas are working plug-in estimates rather than exact conditional identities.

Value

An object of class `"local_fofr_fit"`. It is a list containing:

<code>call</code>	The matched function call.
<code>gamma_hat</code>	The columnwise vectorization of the fitted coefficient matrix, of length $(PM_x + Q)M_y$.
<code>expected_gamma</code>	The plug-in conditional-mean vector used in the feasible transfer summary.
<code>variance_gamma</code>	The spectrally regularized estimated covariance matrix of <code>gamma_hat</code> .
<code>Gamma_matrix</code>	The fitted $(PM_x + Q) \times M_y$ coefficient matrix.
<code>y_mean_curve</code>	The study-level mean response on <code>y_grid</code> .
<code>x_mean_curves</code>	A list containing the study-level mean curve of every functional predictor.
<code>w_means</code>	The scalar-covariate means, or a numeric vector of length zero when the model has no scalar covariates.
<code>y_grid</code>	The response observation grid.
<code>x_grids</code>	The functional-predictor observation grids.
<code>P_x</code>	A list of predictor smoothing matrices, one for each functional predictor.
<code>Phi_x</code>	A list of predictor-basis evaluation matrices on the observed predictor grids.
<code>P_y</code>	The response smoothing matrix.
<code>Theta</code>	The response-basis evaluation matrix on <code>y_grid</code> .
<code>x_basis</code>	The predictor basis supplied in <code>x_basis</code> .
<code>y_basis</code>	The response basis supplied in <code>y_basis</code> .

rho_x	The predictor smoothing penalties actually used.
rho_y	The response smoothing penalty actually used.
lambda	The coefficient-surface roughness penalty actually used.
lambda_selection	A list with components value, grid, and gcv. When lambda was supplied, grid contains that value and gcv is NA_real_.
variance_method	The covariance estimation method.
measurement_variances	The mean squared predictor-reconstruction residual for each functional predictor.
effective_df	The effective degrees of freedom $\text{tr}(\mathbf{A}^{-1}\mathbf{\Omega})$.
design_rank	The numerical rank of the mixed-predictor design matrix.
design_condition	The condition number of the design matrix, or Inf when it is rank deficient.
predictor_names	The ordered functional-predictor names.
covariate_names	The ordered scalar-covariate names.
basis_signature	A compact record of the basis types, dimensions, boundaries, knots, Gram and penalty matrices, and ordered predictor and covariate names. It is used to verify compatibility across studies.
n	The number of subjects in the study.
P_number	The number of functional predictors.
Q_number	The number of scalar covariates.
Mx	The predictor-basis dimension.
My	The response-basis dimension.
L	The number of rows in the coefficient matrix, $PM_x + Q$.

Author(s)

Ufuk Beyaztas, Muge Mutis, Sude Gurer and Soutir Bandyopadhyay.

Examples

```

s_grid <- seq(0, 1, length.out = 101)
t_grid <- seq(0, 1, length.out = 101)
study_data <- simulate_fofr_data(50, s_grid = s_grid, t_grid = t_grid)

x_basis <- fofr_basis("bspline", M = 5)
y_basis <- fofr_basis("bspline", M = 5)

local_fit <- fit_local_fofr(
  dataset = study_data,
  x_basis = x_basis,
  y_basis = y_basis,
  variance_method = "model"

```

```

)

dim(local_fit$Gamma_matrix)
local_fit$lambda
local_fit$effective_df
c(rank = local_fit$design_rank, columns = local_fit$L)

# This is the object transmitted by a source site.
site_summary <- fofr_site_summary(local_fit)
names(site_summary)

```

fofr_basis

*Construct a Basis for Function-on-Function Regression***Description**

Constructs a normalized Fourier or B-spline basis together with its numerical Gram matrix and integrated second-derivative penalty matrix. The resulting object supplies the common functional geometry used by local FoFR fitting, CVS aggregation, and Gram-metric pCVS regularization.

Usage

```

fofr_basis(
  type = c("fourier", "bspline"),
  M = 15,
  degree = 3,
  boundary = c(0, 1),
  integration_grid = seq(boundary[1], boundary[2], length.out = 1001)
)

```

Arguments

type	The basis family: "fourier" or "bspline". Partial matching is supported.
M	The number of basis functions. It must be a single integer of at least 3. For a B-spline basis, it must also satisfy $M \geq \text{degree} + 1$.
degree	The polynomial degree of a B-spline basis. It must be a single integer of at least 2. The value is stored but does not affect a Fourier basis.
boundary	A length-two increasing numeric vector defining the functional domain.
integration_grid	A strictly increasing numeric vector of at least three finite points lying within boundary. It is used for trapezoidal approximation of the Gram and roughness matrices. For a B-spline basis, it is also the reference grid used to place the interior knots.

Details

Let $\phi(u) = (\phi_1(u), \dots, \phi_M(u))^T$ denote the constructed basis on $[a, b]$, where a and b are the entries of boundary. The function returns numerical approximations to

$$\Psi = \int_a^b \phi(u) \phi(u)^T du, \quad \mathbf{W} = \int_a^b \phi''(u) \phi''(u)^T du.$$

For a coefficient vector $\mathbf{c}$ representing $f(u) = \phi(u)^\top \mathbf{c}$, these matrices satisfy

$$\int_a^b f(u)^2 du = \mathbf{c}^\top \Psi \mathbf{c}, \quad \int_a^b \{f''(u)\}^2 du = \mathbf{c}^\top \mathbf{W} \mathbf{c}.$$

Thus, Ψ represents the L^2 geometry of the basis and $\mathbf{W}$ represents second-derivative roughness. For a tensor-product coefficient surface $\beta(s, t) = \phi(s)^\top \Gamma \boldsymbol{\theta}(t)$, the corresponding integrated squared norm is

$$\iint \beta(s, t)^2 ds dt = \text{tr}(\Gamma^\top \Psi_x \Gamma \Psi_y).$$

This identity is the basis of the Gram metric used to compare entire coefficient objects in pCVS.

For type = "fourier", put $D = b - a$ and $v = (u - a)/D$. The basis is ordered as

$$\phi_1(u) = D^{-1/2}, \quad \phi_{2r}(u) = (2/D)^{1/2} \cos(2\pi r v), \quad \phi_{2r+1}(u) = (2/D)^{1/2} \sin(2\pi r v),$$

for the frequencies represented among the first M functions. This normalization makes the continuous Gram matrix the identity. The constant function has zero second-derivative penalty, while the trigonometric penalties increase with the fourth power of frequency.

For type = "bspline", the function constructs an intercept-containing B-spline basis with M functions and the requested polynomial degree. Interior knots are those selected by `splines::bs` from `integration_grid`; the full knot sequence repeats each boundary degree + 1 times. The same knot sequence is subsequently used to evaluate the basis and its second derivatives on study-specific observation grids.

Both Ψ and $\mathbf{W}$ are computed by trapezoidal quadrature on `integration_grid` and are explicitly symmetrized. Increasing the grid density generally improves their approximation to the continuous integrals. For Fourier bases, the basis functions themselves do not depend on this grid. For B-spline bases, changing a nonuniform integration grid may also change the automatically placed interior knots and therefore changes the basis itself.

Value

An object of class "fofr_basis". It is a list containing:

type	The selected basis family.
M	The number of basis functions.
degree	The supplied polynomial degree.
boundary	The two endpoints of the functional domain.
integration_grid	The grid used for knot placement and quadrature.
knots	For a B-spline basis, the interior knots returned by <code>splines::bs</code> . This component is absent for a Fourier basis.
full_knots	For a B-spline basis, the complete knot sequence used for basis evaluation, including repeated boundary knots. This component is absent for a Fourier basis.
Psi	The symmetric $M \times M$ numerical Gram matrix.
W	The symmetric $M \times M$ integrated second-derivative penalty matrix.

Author(s)

Ufuk Beyaztas, Muge Mutis, Sude Gurer and Soutir Bandyopadhyay.

Examples

```

fourier_basis <- fofr_basis("fourier", M = 7)
bspline_basis <- fofr_basis("bspline", M = 7, degree = 3)

dim(fourier_basis$Psi)
round(fourier_basis$Psi, 4)
round(diag(fourier_basis$W), 3)

bspline_basis$knots
range(eigen(
  bspline_basis$Psi,
  symmetric = TRUE,
  only.values = TRUE
)$values)

stopifnot(
  isSymmetric(fourier_basis$Psi),
  isSymmetric(fourier_basis$W),
  isSymmetric(bspline_basis$Psi),
  isSymmetric(bspline_basis$W)
)

```

fofr_site_summary

Create a Transfer Summary from a Local FoFR Fit

Description

Extracts the aggregate coefficient, moment, and basis information required to use a locally fitted function-on-function regression model as a source in the CVS and pCVS transfer procedures. Subject-level responses, predictors, and covariates are not included in the returned object.

Usage

```
fofr_site_summary(local_fit)
```

Arguments

`local_fit` An object of class "local_fofr_fit" produced by [fit_local_fofr](#).

Details

The function defines the communication boundary between a source site and the coordinating target site. Let P be the number of functional predictors, Q the number of scalar covariates, and M_x and M_y the predictor- and response-basis dimensions. The local coefficient matrix has dimension $(PM_x + Q) \times M_y$, and the length of its columnwise vectorization is

$$d = (PM_x + Q)M_y.$$

For study k , the exported numerical summary consists of

$$\mathcal{S}_k = \left(\hat{g}_k, \hat{\mu}_k, \hat{\Sigma}_k \right),$$

where $\hat{\mathbf{g}}_k = \text{vec}(\hat{\mathbf{\Gamma}}_k)$ is the local coefficient vector, $\hat{\boldsymbol{\mu}}_k$ is its feasible plug-in conditional-mean vector, and $\hat{\boldsymbol{\Sigma}}_k$ is the selected plug-in covariance estimate. The mean and covariance are the objects stored as `expected_gamma` and `variance_gamma` by `fit_local_fofr`. In particular, `expected_gamma` should not in general be interpreted as an exact conditional expectation; its exactness depends on the working model and on how the smoothing parameters are treated.

In addition to these numerical summaries, the returned object records the basis dimensions and a basis signature. The signature encodes the predictor and response basis definitions, their Gram and roughness matrices, and the ordered functional-predictor and scalar-covariate names. These checks are essential: target–source coefficient subtraction is meaningful only when all sites use the same coefficient coordinates and the same ordering of model terms. Observation grids may differ across sites, provided that they lie within the common basis boundaries and the fitted models share an identical basis signature.

The numerical communication burden is $2d + d^2$ values—two vectors of length d and one $d \times d$ covariance matrix—apart from the small dimension and signature metadata. It is therefore independent of the number of subjects at the source site. The object does not contain the raw response matrix, functional-predictor matrices, scalar covariates, subject-specific residuals, observation grids, smoothing operators, or tuning paths.

This function performs no cross-study aggregation and does not itself compute a transfer estimator. A list of source summaries is supplied to `combine_cvs_fofr`, together with the target-site local fit, to construct the CVS estimator and the pCVS path.

Value

An object of class "cvs_fofr_summary". It is a list containing:

<code>gamma_hat</code>	The local coefficient vector $\hat{\mathbf{g}} = \text{vec}(\hat{\mathbf{\Gamma}})$, of length d .
<code>expected_gamma</code>	The feasible plug-in conditional-mean vector of length d used to center the target–source coefficient differences.
<code>variance_gamma</code>	The spectrally regularized $d \times d$ plug-in covariance estimate for <code>gamma_hat</code> . Its construction is determined by the <code>variance_method</code> used in <code>fit_local_fofr</code> .
<code>Mx</code>	The predictor-basis dimension M_x .
<code>My</code>	The response-basis dimension M_y .
<code>P_number</code>	The number P of functional predictors.
<code>Q_number</code>	The number Q of scalar covariates.
<code>basis_signature</code>	Metadata identifying the common predictor and response bases and the ordered model terms. It is used by <code>combine_cvs_fofr</code> to reject incompatible source summaries.

Author(s)

Ufuk Beyaztas, Muge Mutis, Sude Gurer and Soutir Bandyopadhyay.

Examples

```
s_grid <- seq(0, 1, length.out = 101)
t_grid <- seq(0, 1, length.out = 101)
source_data <- simulate_fofr_data(
  60,
```

```

    s_grid = s_grid,
    t_grid = t_grid
  )

  x_basis <- fofr_basis("bspline", M = 5)
  y_basis <- fofr_basis("bspline", M = 5)

  source_fit <- fit_local_fofr(
    dataset = source_data,
    x_basis = x_basis,
    y_basis = y_basis,
    variance_method = "model"
  )

  source_summary <- fofr_site_summary(source_fit)

  class(source_summary)
  names(source_summary)

  d <- (source_summary$P_number * source_summary$Mx +
    source_summary$Q_number) * source_summary$My
  c(
    coefficient_length = length(source_summary$gamma_hat),
    expected_length = length(source_summary$expected_gamma),
    theoretical_length = d
  )
  dim(source_summary$variance_gamma)

  # No subject-level observations are retained.
  intersect(c("Y", "X_list", "w", "x_grids", "y_grid"),
    names(source_summary))

```

predict.cvs_fofr_fit *Predict Response Curves from a CVS-FoFR Fit*

Description

Computes fitted response curves for new functional and scalar predictors from the Target-only, control-variates (CVS), or penalized control-variates (pCVS) estimator in a fitted function-on-function transfer model.

Usage

```

## S3 method for class cvs_fofr_fit
predict(
  object,
  newdata,
  method = c("local", "CVS", "pCVS"),
  ...
)

```

Arguments

object	An object of class "cvs_fofr_fit", as returned by <code>fit_cvs_fofr</code> or <code>combine_cvs_fofr</code> .
newdata	A list containing <code>X_list</code> , the new functional-predictor matrices. The list may also contain <code>x_grids</code> , <code>w</code> , and <code>y_grid</code> ; see Details. The scalar-covariate matrix <code>w</code> is required when the fitted model contains scalar covariates.
method	A character string specifying the estimator used for prediction. "local" uses the Target-only estimator, "CVS" uses the feasible control-variates estimator, and "pCVS" uses the selected penalized control-variates estimator. Partial matching is supported.
...	Additional arguments for the predict generic. They are currently unused.

Details

Suppose the fitted model contains P functional predictors and Q scalar covariates. For a new subject i , let $\hat{\mathbf{h}}_{i0}$ be the mixed-predictor design vector formed using the target-site centering and smoothing quantities. For estimator $m \in \{\text{local}, \text{CVS}, \text{pCVS}\}$, the predicted curve is

$$\hat{Y}_i^{(m)}(t) = \bar{Y}_0(t) + \hat{\mathbf{h}}_{i0}^T \hat{\mathbf{\Gamma}}^{(m)} \boldsymbol{\vartheta}(t),$$

where $\bar{Y}_0(t)$ is the target training-sample mean response, $\boldsymbol{\vartheta}(t)$ is the fitted response basis, and $\hat{\mathbf{\Gamma}}^{(m)}$ is the coefficient matrix selected by method. Thus, transfer modifies the centered regression effects; all three prediction rules remain anchored by the target-site mean response.

For functional predictor p , the corresponding block of $\hat{\mathbf{h}}_{i0}$ is $\hat{\mathbf{c}}_{ip,0}^T \boldsymbol{\Psi}_x$, where $\hat{\mathbf{c}}_{ip,0}$ is obtained after subtracting the target mean predictor curve and smoothing on the new predictor grid with the target-fitted basis and smoothing penalty. The scalar block is $(\mathbf{w}_i - \bar{\mathbf{w}}_0)^T$. In particular, method = "local" means prediction with the Target-only coefficient estimator; it does not refit the model to newdata.

The component `X_list` must contain the same number of functional predictors used to fit object, with a common number of rows. Each matrix has one row per new subject and one column per location in the corresponding grid. A vector is accepted as a single-subject predictor, and a matrix may be supplied directly when the model has one functional predictor. If `X_list` is fully named, its names must match the fitted predictor names and its elements are reordered accordingly; otherwise its order is assumed to match the fitted model. All predictor values must be finite.

The optional `x_grids` component supplies one finite, strictly increasing grid per functional predictor. Its names, when complete, must match the fitted predictor names. If `x_grids` is omitted, the target training grids are used. New predictor grids may differ from the training grids but must remain within their respective target training-grid ranges; extrapolation is not performed. The number of columns of each predictor matrix must equal the length of its grid.

When $Q > 0$, `w` must be a finite matrix with one row per new subject and Q columns. A vector is reshaped to have the same number of rows as the functional predictors and must therefore have compatible length. Fully specified column names must match the fitted scalar covariate names, in which case the columns are placed in the fitted order. The component is unnecessary when the model has no scalar covariates.

The optional `y_grid` determines the response locations at which the predictions are evaluated. It must contain at least two finite, strictly increasing values within the target response training-grid range. If it is omitted, the target training response grid is used. On a different admissible grid, the response basis is evaluated anew and the target mean response is linearly interpolated. A response matrix `Y` may be present in newdata, for example when a complete test data set is supplied, but it is not used in forming the predictions.

For method = "CVS", the coefficient matrix is obtained from the feasible control-variates adjustment of the Target-only estimator. For method = "pCVS", it is the selected member of the group-penalized transfer path. A pCVS prediction is available only after a penalty has been selected by a target validation sample or supplied through zeta; an error is returned otherwise.

Value

A numeric matrix with one row for each new subject and one column for each location in the selected response grid. If `newdata$y_grid` is omitted, the number of columns equals the length of the target training response grid. Entry `[i, j]` is the predicted response for subject `i` at the `j`th response-grid location.

Author(s)

Ufuk Beyaztas, Muge Mutis, Sude Gurer and Soutir Bandyopadhyay.

Examples

```
s_train <- seq(0, 1, length.out = 101)
t_train <- seq(0, 1, length.out = 101)
s_test  <- seq(0, 1, length.out = 101)
t_test  <- seq(0, 1, length.out = 101)

target <- simulate_fofr_data(40, s_grid = s_train, t_grid = t_train)
source <- simulate_fofr_data(80, s_grid = s_train, t_grid = t_train)
validation <- simulate_fofr_data(40, s_grid = s_train, t_grid = t_train)
test <- simulate_fofr_data(50, s_grid = s_test, t_grid = t_test)

fit <- fit_cvs_fofr(
  datasets = list(target, source),
  x_basis_type = "bspline",
  y_basis_type = "bspline",
  Mx = 5,
  My = 5,
  variance_method = "model",
  target_validation = validation
)

prediction_data <- list(
  X_list = test$X_list,
  x_grids = test$x_grids,
  w = test$w,
  y_grid = test$y_grid
)

pred_target <- predict(fit, prediction_data, method = "local")
pred_cvs <- predict(fit, prediction_data, method = "CVS")
pred_pcv <- predict(fit, prediction_data, method = "pCVS")

dim(pred_pcv)
pred_pcv[1, 1:5]

# Integrated squared prediction error on a test sample.
response_weights <- c(0.5, rep(1, length(t_test) - 2), 0.5) *
```



```
diff(range(t_test)) / (length(t_test) - 1)
mean(as.numeric((test$Y - pred_pcv_s)^2 %*% response_weights))
```

```
print.cvs_fofr_fit      Print a CVS-FoFR Fit
```

Description

Prints a compact summary of a fitted function-on-function control-variates transfer model, including model dimensions, target-site regularization and design diagnostics, and the status of pCVS penalty selection.

Usage

```
## S3 method for class cvs_fofr_fit
print(x, ...)
```

Arguments

x An object of class "cvs_fofr_fit", as returned by [fit_cvs_fofr](#) or [combine_cvs_fofr](#).
... Additional arguments for the print generic. They are currently unused.

Details

The method reports the target sample size; the numbers of functional predictors, scalar covariates, and source studies; and the predictor- and response-basis dimensions M_x and M_y . If there are P functional predictors and Q scalar covariates, the mixed-predictor design has

$$L = PM_x + Q$$

columns, while the vectorized coefficient object has dimension LM_y .

The displayed target value of lambda is the roughness penalty used in the local target fit. It is either the user-supplied value or the value chosen by generalized cross-validation in [fit_local_fofr](#). The target design rank is the numerical rank of the centered mixed-predictor design matrix, reported relative to its L columns. The condition number is the ratio of the largest to the smallest retained singular value when the design has full column rank; it is reported as infinite for a rank-deficient design. These quantities diagnose numerical conditioning and identifiability of the local coefficient blocks. They are not measures of predictive performance.

If no pCVS penalty has been selected, the method prints ‘pCVS: no zeta selected’. This occurs when neither a user-specified zeta nor a target validation sample was supplied to the combining procedure. In that case, the Target-only and CVS estimators and the fitted pCVS path remain available, but there is no selected pCVS coefficient vector for extraction or prediction.

When a pCVS estimator has been selected, the method reports its penalty ζ , the selection rule, and the relative Karush–Kuhn–Tucker (KKT) residual of the selected path solution. The pCVS groups are regularized in the Gram geometry of the coefficient functions. If Ψ_x and Ψ_y are the predictor and response Gram matrices, this geometry is induced by

$$G = \Psi_y \otimes \text{diag}(\Psi_x, \dots, \Psi_x, I_Q).$$

Consequently, the group norm corresponds to the integrated L^2 norm of the collection of coefficient surfaces and scalar-effect functions rather than an unscaled Euclidean norm of basis coefficients.

The relative KKT residual is a dimensionless numerical optimality diagnostic; smaller values indicate closer satisfaction of the first-order conditions. It should not be interpreted as an estimation error, a test statistic, or a measure of model fit.

The output is intentionally selective. Coefficient estimates, validation losses, the complete pCVS path, convergence records, and source-specific discrepancies are stored in `x` but are not printed by this method.

Value

The object `x`, returned invisibly. The method is called for its printed side effect.

Author(s)

Ufuk Beyaztas, Muge Mutis, Sude Gurer and Soutir Bandyopadhyay.

Examples

```
s_grid <- seq(0, 1, length.out = 101)
t_grid <- seq(0, 1, length.out = 101)

target <- simulate_fofr_data(35, s_grid = s_grid, t_grid = t_grid)
source <- simulate_fofr_data(60, s_grid = s_grid, t_grid = t_grid)
validation <- simulate_fofr_data(35, s_grid = s_grid, t_grid = t_grid)

fit <- fit_cvs_fofr(
  datasets = list(target, source),
  x_basis_type = "bspline",
  y_basis_type = "bspline",
  Mx = 5,
  My = 5,
  variance_method = "model",
  target_validation = validation
)

print(fit)

returned_object <- print(fit)
identical(returned_object, fit)
```

scalar_effect

Evaluate a Scalar Effect from a CVS-FoFR Fit

Description

Reconstructs the response-varying coefficient function associated with one scalar covariate from the Target-only, control-variates (CVS), or penalized control-variates (pCVS) estimator in a fitted function-on-function transfer model.

Usage

```
scalar_effect(
  object,
  covariate = 1,
  method = c("local", "CVS", "pCVS"),
  t_grid = NULL
)
```

Arguments

object	An object of class "cvs_fofr_fit", as returned by <code>fit_cvs_fofr</code> or <code>combine_cvs_fofr</code> .
covariate	The scalar covariate whose coefficient function is to be evaluated. Supply either a single integer between 1 and the number of scalar covariates or the exact covariate name recorded in object.
method	A character string specifying the estimator. "local" uses the Target-only estimator, "CVS" uses the feasible control-variates estimator, and "pCVS" uses the selected penalized control-variates estimator. Partial matching is supported.
t_grid	An optional numeric vector of response-domain locations at which the scalar effect is evaluated. If NULL, 201 equally spaced points over the boundary of the fitted response basis are used.

Details

For a function-on-function regression model with P functional predictors and Q scalar covariates, the conditional mean has the form

$$\mathbb{E}\{Y_i(t) \mid X_{i1}, \dots, X_{iP}, \mathbf{w}_i\} = \mu_0(t) + \sum_{p=1}^P \int_{\mathcal{S}} X_{ip}(s) \beta_p(s, t) ds + \sum_{q=1}^Q w_{iq} \alpha_q(t).$$

The function $\alpha_q(t)$ is therefore the change in the conditional mean response at location t associated with a one-unit increase in scalar covariate q , holding the other predictors fixed. The fitted model is estimated after within-study centering, but centering changes the functional intercept rather than the coefficient function. Hence the same slope interpretation applies relative to the target covariate mean.

Let M_x and M_y denote the predictor- and response-basis dimensions and put $L = PM_x + Q$. The fitted coefficient matrix $\hat{\mathbf{\Gamma}}$ has L rows. Its first PM_x rows contain the coefficient blocks of the functional predictors, and its final Q rows contain the scalar-effect coefficients. If $\hat{\gamma}_{wq}^T$ is the row associated with covariate q and $\boldsymbol{\vartheta}(t)$ is the response basis, the reconstructed effect is

$$\hat{\alpha}_q(t) = \hat{\gamma}_{wq}^T \boldsymbol{\vartheta}(t).$$

The function selects the requested coefficient vector, reshapes it columnwise into $\hat{\mathbf{\Gamma}}$, extracts row $PM_x + q$, and evaluates this expression on `t_grid`.

For `method = "local"`, the result is obtained from the penalized Target-only fit. For `method = "CVS"`, it uses the feasible control-variates adjustment based on the centered target–source coefficient differences. For `method = "pCVS"`, it uses the selected point on the group-penalized transfer path. The pCVS discrepancy penalty acts jointly on all coefficient surfaces and scalar-effect functions contributed by a source; it does not select scalar effects separately. Within the coefficient Gram metric, the scalar part uses the response Gram matrix, so its squared norm is the integrated squared magnitude of $\alpha_q(t)$.

The supplied `t_grid` is an evaluation grid only: changing it does not refit the model. For interpretation on the fitted response domain, its values should lie within the boundary of the response basis. If the model contains no scalar covariates, the function returns an error. Extraction with `method = "pCVS"` requires that a pCVS penalty has been selected by a target validation sample or supplied through `zeta`; otherwise an error is returned.

Value

A data frame with one row per response-domain evaluation location and the following columns:

<code>t_grid</code>	The response-domain evaluation locations.
<code>alpha</code>	The reconstructed scalar coefficient function.
<code>method</code>	The estimator used for reconstruction.
<code>covariate</code>	The numerical index of the selected scalar covariate.

Author(s)

Ufuk Beyaztas, Muge Mutis, Sude Gurer and Soutir Bandyopadhyay.

Examples

```
s_grid <- seq(0, 1, length.out = 101)
t_grid <- seq(0, 1, length.out = 101)

target <- simulate_fofr_data(40, s_grid = s_grid, t_grid = t_grid)
source <- simulate_fofr_data(80, s_grid = s_grid, t_grid = t_grid)
validation <- simulate_fofr_data(40, s_grid = s_grid, t_grid = t_grid)

fit <- fit_cvs_fofr(
  datasets = list(target, source),
  x_basis_type = "bspline",
  y_basis_type = "bspline",
  Mx = 5,
  My = 5,
  variance_method = "model",
  target_validation = validation
)

continuous_effect <- scalar_effect(
  fit,
  covariate = "continuous",
  method = "pCVS",
  t_grid = t_grid
)

head(continuous_effect)

plot(
  t_grid,
  target$truth$alpha$continuous,
  type = "l",
  lwd = 2,
  xlab = "t",
```

```

    ylab = expression(alpha[1](t))
  )
  lines(
    continuous_effect$t_grid,
    continuous_effect$alpha,
    col = "red",
    lwd = 2
  )
  legend(
    "topright",
    legend = c("Truth", "pCVS"),
    col = c("black", "red"),
    lwd = 2,
    bty = "n"
  )

```

simulate_fofr_data

*Simulate Mixed-Predictor Function-on-Function Data***Description**

Generates a functional response, two noisy functional predictors, and two scalar covariates from the mixed-predictor function-on-function regression model used in the package examples. Common and shifted regression-effect regimes are available for illustrating transferable and nontransferable source studies.

Usage

```

simulate_fofr_data(
  n,
  s_grid = seq(0, 1, length.out = 101),
  t_grid = seq(0, 1, length.out = 101),
  covariance_scale = 4,
  effect = c("common", "shifted"),
  measurement_sd = 0.05,
  error_sd = 0.10
)

```

Arguments

<code>n</code>	A single integer of at least 5 giving the number of subjects.
<code>s_grid</code>	A numeric vector containing at least two finite, strictly increasing predictor-domain locations.
<code>t_grid</code>	A numeric vector containing at least two finite, strictly increasing response-domain locations.
<code>covariance_scale</code>	A positive finite number multiplying the covariance kernel of both latent functional predictors. Changing this value alters the predictor distribution without changing the regression effects.

effect	The regression-effect regime. "common" uses the reference coefficient functions; "shifted" changes both function-on-function surfaces and both scalar-effect functions. Partial matching is supported.
measurement_sd	A nonnegative finite number giving the standard deviation of independent measurement error added to each latent functional-predictor value.
error_sd	A nonnegative finite number giving the standard deviation of independent response error at each response-grid location.

Details

For each subject i , two independent latent predictor curves, X_{iA}° and X_{iE}° , are generated as mean-zero Gaussian processes. On `s_grid`, both have covariance kernel

$$C(s, u) = a \exp\{-10|s - u|\},$$

where a is `covariance_scale`. The implementation adds 10^{-9} to the diagonal of the discretized covariance matrix before its Cholesky factorization. The two scalar covariates are generated independently as

$$w_{i1} \sim N(0, 1), \quad w_{i2} \sim \text{Bernoulli}(0.5).$$

The observed functional predictors are

$$Z_{ip}(s_j) = X_{ip}^\circ(s_j) + e_{ipj}, \quad e_{ipj} \sim N(0, \sigma_m^2),$$

for $p \in \{A, E\}$, where σ_m is `measurement_sd`. Measurement errors are independent across subjects, predictors, and grid locations and are independent of the latent curves and scalar covariates.

The functional response is generated from

$$Y_i(t) = \int_S X_{iA}^\circ(s) \beta_A(s, t) ds + \int_S X_{iE}^\circ(s) \beta_E(s, t) ds + w_{i1} \alpha_1(t) + w_{i2} \alpha_2(t) + \varepsilon_i(t).$$

The integrals are evaluated numerically on `s_grid` using trapezoidal weights. The response errors are independent $N(0, \sigma_\varepsilon^2)$ variables across subjects and response-grid locations, with σ_ε equal to `error_sd`. The returned `conditional_mean` is the noise-free mean conditional on the latent predictor curves and scalar covariates, rather than on their measurement-error-contaminated observations.

Under `effect = "common"`, the coefficient functions are

$$\beta_A(s, t) = \sin(2\pi s) \cos(\pi t), \quad \beta_E(s, t) = 2(s - 0.5)(t - 0.5),$$

and

$$\alpha_1(t) = 1 + 0.5 \sin(2\pi t), \quad \alpha_2(t) = -0.8t.$$

Under `effect = "shifted"`, they become

$$\beta_A^{\text{shift}}(s, t) = -0.35 \beta_A(s, t),$$

$$\beta_E^{\text{shift}}(s, t) = \beta_E(s, t) + 1.25 \cos(\pi s) \sin(2\pi t),$$

and

$$\alpha_1^{\text{shift}}(t) = -0.50 \alpha_1(t), \quad \alpha_2^{\text{shift}}(t) = -0.8t + 0.9 \sin(\pi t).$$

Thus, independently generated studies with `effect = "common"` share the same regression operator and are transferable under the simulation model. Changing only `covariance_scale` creates predictor-covariance shift while preserving that operator. By contrast, `effect = "shifted"` creates a source with a genuinely different regression operator and is intended for illustrating possible negative transfer and the protective role of pCVS.

Value

A list containing:

<code>Y</code>	An $n \times J_t$ matrix of observed response curves, where J_t is the length of <code>t_grid</code> .
<code>y_grid</code>	The response grid supplied in <code>t_grid</code> .
<code>X_list</code>	A named list with elements <code>activity</code> and <code>exposure</code> . Each is an $n \times J_s$ matrix of measurement-error-contaminated functional predictors, where J_s is the length of <code>s_grid</code> .
<code>x_grids</code>	A named list assigning <code>s_grid</code> to both functional predictors.
<code>w</code>	An $n \times 2$ matrix with columns <code>continuous</code> and <code>binary</code> .
<code>conditional_mean</code>	An $n \times J_t$ matrix containing the noise-free response means conditional on the latent functional predictors and scalar covariates.
<code>latent_X_list</code>	A named list containing the latent, measurement-error-free <code>activity</code> and <code>exposure</code> predictor matrices.
<code>truth</code>	A nested list. <code>truth\$beta</code> contains the $J_s \times J_t$ matrices <code>activity</code> and <code>exposure</code> ; <code>truth\$alpha</code> contains the length- J_t vectors <code>continuous</code> and <code>binary</code> .
<code>effect</code>	The selected regression-effect regime.
<code>covariance_scale</code>	The predictor-covariance scale used to generate the latent curves.

Author(s)

Ufuk Beyaztas, Muge Mutis, Sude Gurer and Soutir Bandyopadhyay.

Examples

```
s_grid <- seq(0, 1, length.out = 101)
t_grid <- seq(0, 1, length.out = 101)

target <- simulate_fofr_data(
  40,
  s_grid = s_grid,
  t_grid = t_grid
)
transferable_source <- simulate_fofr_data(
  80,
  s_grid = s_grid,
  t_grid = t_grid,
  covariance_scale = 7
)
shifted_source <- simulate_fofr_data(
  80,
  s_grid = s_grid,
  t_grid = t_grid,
  effect = "shifted"
)
validation <- simulate_fofr_data(
  40,
  s_grid = s_grid,
```

```
    t_grid = t_grid
  )

  dim(target$Y)
  names(target$X_list)
  names(target$truth$beta)
  mean((target$Y - target$conditional_mean)^2)

  fit <- fit_cvs_fofr(
    datasets = list(target, transferable_source, shifted_source),
    x_basis_type = "bspline",
    y_basis_type = "bspline",
    Mx = 5,
    My = 5,
    variance_method = "model",
    target_validation = validation
  )

  fit
```


Index

`coef.cvs_fofr_fit`, [2](#)
`coefficient_surface`, [3](#), [4](#)
`combine_cvs_fofr`, [2](#), [4](#), [6](#), [12](#), [13](#), [21](#), [23](#), [25](#),
[27](#)

`fit_cvs_fofr`, [2](#), [4](#), [10](#), [23](#), [25](#), [27](#)
`fit_local_fofr`, [6](#), [14](#), [20](#), [21](#), [25](#)
`fofr_basis`, [14](#), [18](#)
`fofr_site_summary`, [6](#), [20](#)

`predict.cvs_fofr_fit`, [22](#)
`print.cvs_fofr_fit`, [25](#)

`scalar_effect`, [3](#), [26](#)
`simulate_fofr_data`, [29](#)